WEEK 1:- Rabbit Leap Problem :- BFS and DFS State space search

Varsha (20261604007), Rahul (20261602015), Sukriti Singh (20261602019)

November 1, 2026

---

1 Problem Statement

The Rabbit Leap problem is a classic state-space search problem. There are 3 white rabbits (>) on the left, 3 black rabbits (<) on the right, and one empty space (_) in the middle.
Initial State: `>>> _ <<<`
Goal State: `<<< _ >>>`

Rabbits can move according to:

- A rabbit can move one step forward into an empty space.
- A rabbit can jump over exactly one rabbit of opposite type into an empty space.
- Rabbits cannot move backwards.

2 State Space Search

2.1 Valid Moves

Let empty position be $i = s.index('_')$.

1. `> moves 1 right:` If $i - 1 \geq 0$ and $s[i-1] == '>'$

2. `< moves 1 left:` If $i + 1 < len(s)$ and $s[i+1] == '<'$

3. `> jumps over <:` If $i - 2 \geq 0$ and $s[i-2] == '>'$ and $s[i-1] == '<'$

4. `< jumps over >:` If $i + 2 < len(s)$ and $s[i+2] == '<'$ and $s[i+1] == '>'$

2.2 Search Space Size

Total combinations $= \frac{7!}{3! \times 3! \times 1!} = 140$ possible states. Branching factor $b \approx 2$. Optimal depth $d = 15$.

3 BFS SOLUTION

3.1 Python Code - BFS

```python
from collections import deque

def next_states(state):
    s = list(state)
    i = s.index('_')
    if i - 1 >= 0 and s[i - 1] == '>':
        t = s.copy(); t[i], t[i - 1] = t[i - 1], t[i]
        yield ('> moves 1 right', ''.join(t))
    if i + 1 < len(s) and s[i + 1] == '<':
        t = s.copy(); t[i], t[i + 1] = t[i + 1], t[i]
        yield ('< moves 1 left', ''.join(t))
    if i - 2 >= 0 and s[i - 2] == '>' and s[i - 1] == '<':
        t = s.copy(); t[i], t[i - 2] = t[i - 2], t[i]
        yield ('> jumps over <', ''.join(t))
    if i + 2 < len(s) and s[i + 2] == '<' and s[i + 1] == '>':
        t = s.copy(); t[i], t[i + 2] = t[i + 2], t[i]
        yield ('< jumps over >', ''.join(t))

def bfs(start=">>>_<<<", goal="<<<_>>>"):
    q = deque([start])
    parent = {start: None}
    action = {}
    seen = {start}
    while q:
        u = q.popleft()
        if u == goal:
            break
        for a, v in next_states(u):
            if v not in seen:
                seen.add(v)
                parent[v] = u
                action[v] = a
                q.append(v)
    if goal not in parent:
        return None
    path = []
    s = goal
    while s is not None:
        path.append(s)
        s = parent[s]
    path.reverse()
    steps = []
    for i in range(1, len(path)):
        steps.append((action[path[i]], path[i]))
    return steps

steps = bfs()
print("BFS found steps:", len(steps))
for move, state in steps:
    print(f"{move:>18} -> {state}")
```

Listing 1: rabit-bfs.py

3.2 Execution Screenshot - BFS



Figure 1: BFS Execution Output - Found 15 steps

4 DFS SOLUTION

4.1 Python Code - DFS

```
# Depth First Search for Rabbit Leap problem

def next_states(state):
    s = list(state)
    i = s.index('_')
    if i - 1 >= 0 and s[i - 1] == '>':
        t = s.copy(); t[i], t[i - 1] = t[i - 1], t[i]
        yield ('> moves 1 right', ''.join(t))
    if i + 1 < len(s) and s[i + 1] == '<':
        t = s.copy(); t[i], t[i + 1] = t[i + 1], t[i]
        yield ('< moves 1 left', ''.join(t))
    if i - 2 >= 0 and s[i - 2] == '>' and s[i - 1] == '<':
        t = s.copy(); t[i], t[i - 2] = t[i - 2], t[i]
        yield ('> jumps over <', ''.join(t))
    if i + 2 < len(s) and s[i + 2] == '<' and s[i + 1] == '>':
        t = s.copy(); t[i], t[i + 2] = t[i + 2], t[i]
        yield ('< jumps over >', ''.join(t))

def dfs(start=">>>_<<<", goal="<<<_>>>"):
    stack = [(start, [])]
    visited = {start}
```

```
    while stack:
        state, path = stack.pop()
        if state == goal:
            return path
        for move, nxt in reversed(list(next_states(state))):
            if nxt not in visited:
                visited.add(nxt)
                stack.append((nxt, path + [(move, nxt)]))
    return None

steps = dfs()
print("DFS found steps:", len(steps))
for move, state in steps:
    print(f"{move:>18} -> {state}")
```

Listing 2: rabit.dfs.py

4.2 Execution Screenshot - DFS



Figure 2: DFS Execution Output - Found 15 steps

5 Comparison Table: BFS vs DFS

Aspect	BFS	DFS
Data Structure	Queue (FIFO) using deque	Stack (LIFO)
Completeness	Yes, complete	Yes, complete for finite space
Optimality	Yes, gives shortest path (15)	Not guaranteed, but here also 15
Time Complexity	$O(b^d)$	$O(b^m)$
Space Complexity	$O(b^d)$ - High memory	$O(b \cdot m)$ - Low memory
Execution Style	Level by level	Goes deep first
Implementation	parent dict for path reconstruction	Direct path in stack

Table 1: Parameter comparison between BFS and DFS

6 Final Comparison and Conclusion

Both BFS and DFS successfully solve the Rabbit Leap Problem in 15 moves, which is the optimal solution for 3 white and 3 black rabbits.

BFS is preferred when optimality is required, as it always finds the shortest path. However, it consumes more memory.

DFS is more memory efficient and can be faster, but may not guarantee optimality in general graphs. In this problem, due to small state space, both performed equally well.

to small state space, both performed equally well.

7 Code Availability

Complete source codes for this lab are available at the following local paths:

VI. CODE AVAILABILITY

- The complete source code is available at: GitHub Repository (CS605 – AI Laboratory).
- **GitHub Repository:** `https://github.com/20261604007-ux/CS-605-Artifical-Intelligence/tree/main/Week-1`